

AULA 11

O mundo, a deriva e o mapa que corrige

Etapa 6 · terça, 29/09/2026 · sala SJ205 · Prof. Dácio Moreira de Souza

A odometria não erra por ruído. Erra por viés.

Ruído se cancela: medir mais vezes ajuda. Viés não — ele entra na conta sempre com o mesmo sinal, e cresce com a distância percorrida.

Cenas dos últimos episódios: onde paramos

- A clínica validou o modelo do robô e a árvore de transformadas até a câmera
- A aterrissagem fechou o trabalho: branch, tag, relatório e Moodle
- O bloco de launch era o de folga declarado — e foi ele que cedeu quando o tempo apertou
- Ele abre hoje, e é a primeira coisa que vocês vão rodar

Quatro peças antigas voltam hoje — tenha as quatro na cabeça

- Aula 2, launch: um arquivo sobe vários nós. Hoje ele sobe o PROJETO INTEIRO
- Aula 6, interfaces: o contrato fica, o miolo troca. Hoje troca o simulador
- Aula 8, ações: objetivo longo, com feedback e cancelamento. Volta no fim da aula
- Aula 10, URDF e TF: a árvore até a câmera. Hoje ela ganha DUAS arestas acima
- E da Aula 9: ruído se combate com repetição, porque ele se cancela

O que você sai sabendo fazer hoje

- Subir o seu projeto inteiro com um comando — e provar que ele subiu
- Dizer quem publica cada aresta da árvore de transformadas do seu robô
- Medir o erro da odometria e explicar por que mais medições não o corrigem
- Dizer o que falta a um robô que apenas reage ao que vê

A aula está ordenada por custo de perda, não por sequência lógica

- 1. Launch do sistema inteiro — a entrega é amanhã — não pode cair
- 2. O mundo como produtor de dados — entrega em 06/10 — não pode cair
- 3. A aresta que corrige, e a deriva — entrega em 10/10 — não pode cair
- 4. O que o Nav2 acrescenta — só no próximo trabalho — volta na Aula 12

O corte vem antes do aperto.

É a disciplina da Aula 9 aplicada ao tempo: a régua vem antes da medida. Escolher de antemão o bloco que cede é decisão; descobrir no fim é acidente.

PARTE 1

Launch: um comando sobe o sistema inteiro

Três mecanismos, e os três reaparecem no seu bringup

- Compor — IncludeLaunchDescription traz o subsistema inteiro, com seus argumentos
- Parametrizar — o YAML vem de fora, então o mesmo launch sobe teste e bancada
- Condicionar — IfCondition liga e desliga pedaços
- O bringup não sabe o que cada nó faz: conhece nome e parâmetros. Contrato, de novo

Suba o exemplo e olhe o log

```
ros2 launch aula10_bringup bringup.launch.py  
ros2 launch aula10_bringup bringup.launch.py modelo:=false
```

Sem câmera e sem hardware. Repare no supervisor reclamando e se recuperando sozinho.

Os três mecanismos, no arquivo

aula10-bringup/launch/bringup.launch.py

```
DeclareLaunchArgument('params', default_value=padrao),

IncludeLaunchDescription(                                # 1. COMPOR
    PythonLaunchDescriptionSource(percepcao),
    launch_arguments={'params': params}.items()),

Node(package='robot_state_publisher', ...,              # 2. CONDICIONAR
    condition=IfCondition(modelo),
    parameters=[{'robot_description': descricao}]),
```

O bringup não repete nós: ele compõe subsistemas. Cada um no seu arquivo, e legível.



Um launch que sobe sem erro não é um launch que funciona.

Subir é o que o launch faz. Funcionar é outra afirmação — e ela precisa de prova.

As três verificações

```
ros2 launch aula10_bringup bringup.launch.py --show-args # o que dá para configurar
ros2 node list                                           # quem realmente subiu
ros2 param dump /percepcao/detector                    # com que valores
```

A terceira é a que quase ninguém faz. É ela que pega o erro do próximo slide.

O YAML que ninguém lê

```
ros2 launch aula10_bringup bringup.launch.py \  
  params:=../config/sistema-armadilha.yaml  
  
ros2 param get /percepcao/detector taxa_hz  
-> 2.0          # o padrão do código, não o 9.0 do arquivo
```

O arquivo pedia 9.0. O nó subiu com 2.0. Nenhum erro foi impresso em lugar nenhum.

A causa: a chave do YAML não casa com o nome do nó

- detector: significa o nó /detector, na raiz
- Mas o launch empurrou o nó para /percepcao/detector
- A regra do ROS 2 é ignorar o que não casa — sem avisar
- Curas: /percepcao/detector: ou /**: — prefira o curinga
- Ele sobrevive à mudança de namespace, que é o que você vai fazer quando o sistema crescer

É a mesma família de falhas que vocês já conhecem

- O `view_frames` que gera PDF vazio quando o launch não está rodando
- A saturação que vira ruído puro quando o valor é quase zero
- O `ros2 action list` que funciona sem `source` — e o `send_goal` que não
- São erros que não se apresentam como erro
- A defesa: verificar a afirmação, em vez de tratar o silêncio como aprovação

O conferidor roda sem ROS 2 instalado

```
ros2 node list | python3 conferir-params.py config/sistema.yaml -
```

```
# OK    /percepcao/detector    casou com: /**
```

```
# NADA /percepcao/detector    nenhuma chave casa com este no
```

Ele compara o YAML com os nomes que o `ros2 node list` imprime. Use antes de perder a tarde.

Sobre ordem: nada espera nada

- O supervisor quase sempre nasce antes do detector, reclama, e se recupera sozinho
- Isso é o comportamento correto, não defeito
- Em ROS 2 o grafo é descoberto, não montado em ordem
- Nó que exige ordem de subida quebra no dia em que a máquina está lenta
- A regra: o nó tolera a ausência do outro

PARTE 2

O mundo é um produtor de dados, não uma janela

O trabalho pede um cenário. Do que o seu sistema realmente depende?

- Não do Gazebo
- De um tópico /scan
- De um tópico /odom
- De uma árvore de TF que ligue os dois ao robô
- Qualquer coisa que produza os três serve — e é por isso que o exemplo de hoje existe

O mundo mínimo: sem física, sem render, sem janela

```
ros2 launch aula11_mundo mundo.launch.py  
ros2 topic Hz /scan  
ros2 topic echo /odom --once
```

12 x 8 metros, duas salas ligadas por uma porta, três obstáculos.

O laser inteiro são duas linhas

[aula11-mundo/aula11_mundo/mapa.py](#)

```
d = np.arange(passo, alcance_max + passo, passo) # (D,)
ang = theta + np.asarray(angulos)                # (N,)
xs = x + np.cos(ang)[: , None] * d[None, :]       # (N, D)
ys = y + np.sin(ang)[: , None] * d[None, :]

bateu = self.grade[i, j] | fora                    # (N feixes, D amostras)
primeiro = bateu.argmax(axis=1)                    # argmax pega o 1o True
```

Cada feixe é uma linha de pontos; a leitura é o primeiro ponto ocupado. Erra meio passo — 1 cm —, menos que o ruído de um laser real.



Isto não é desistir do Gazebo. É chegar nele com o esqueleto pronto.

Troque o nó mundo por um Gazebo publicando nos mesmos tópicos e todo o resto continua valendo: o launch, os frames, o SLAM, a navegação.

Quem publica cada aresta — e saber isto é metade do trabalho

- `base_footprint` -> `camera_link` · `robot_state_publisher`, lendo o seu URDF
- `base_footprint` -> `laser_frame` · transformada estática: o sensor não se mexe
- `odom` -> `base_footprint` · o nó mundo, integrando a odometria
- `map` -> `odom` · hoje, identidade fixa — um PLACEHOLDER
- E a aresta de cima de tudo está, neste momento, mentindo

PARTE 3

map -> odom: a aresta que corrige

Veja o número crescer

```
ros2 launch aula11_mundo mundo.launch.py
ros2 topic echo /deriva          # noutro terminal

ros2 launch aula11_mundo mundo.launch.py deriva:=0.0    # e compare
```

/deriva é a distância entre onde o robô ESTÁ e onde a odometria ACHA que ele está.

A aula inteira cabe em três linhas

[aula11-mundo/aula11_mundo/nucleo.py](#)

```
def aplicar_deriva(pose_odom, v, w, dt, deriva_pct):  
    k = 1.0 + deriva_pct / 100.0  
    return passo_diferencial(pose_odom, v * k, w * k, dt)
```

O k é CONSTANTE. É isso que faz dele viés e não ruído — e integrar erro de sinal constante é somar, não cancelar.

O modelo erra de um jeito específico, e o jeito importa

- Erro de escala de 3% — o de quem mediu o raio da roda com régua
- Não é aleatório: tem sinal constante
- Ruído você combate com repetição, porque ele se cancela
- Foi por isso que a Aula 9 usou mediana em vez de uma medida só
- Viés não se cancela nunca: cresce com a distância percorrida. Rodar mais tempo piora

Nenhum filtro sobre /odom resolve.

O conserto não está em medir melhor a roda. Está em medir contra outra coisa — algo que não se mexe.

É isso que o SLAM faz, e é literalmente o trabalho dele

- Compara o /scan de agora com o mapa que vem construindo
- Calcula quanto a odometria já errou
- Publica essa correção como map -> odom
- Enquanto essa aresta for identidade, o robô acredita na própria odometria
- E no RViz2 você vê o laser deslizar para fora das paredes

Noventa por cento dos problemas de SLAM e de navegação são problemas de TF disfarçados.

Se o `view_frames` mostrar árvore quebrada ou dois `odom`, pare tudo e conserte antes de tocar no mapeamento. Insistir com TF errada é queimar uma semana.

PARTE 4

O que o Nav2 acrescenta

O piloto do exemplo é o navegador mais burro que funciona

- Anda reto. Gira quando fecha na frente. Trinta linhas
- E explora o mundo inteiro sozinho
- Repare no que ele NÃO tem — é essa lista, e não outra coisa, que o Nav2 é

Cada falta do piloto vira um pedaço do Nav2

- Não sabe onde está -> localização (AMCL) contra o mapa
- Não sabe para onde vai -> um objetivo, enviado como action
- Não vê além do laser -> custos global e local, construídos do mapa
- Não se recupera ao travar -> comportamentos de recuperação

O objetivo do Nav2 é uma action.

Com feedback periódico e cancelamento — exatamente as da Aula 8. O Nav2 não é assunto novo: é a montagem de peças que já estão no lugar.

Tarefa da semana: o seu projeto sobe inteiro com um comando

- O QUE FAZER: um ros2 launch que sobe os seus nós, publica /scan e /odom, e fecha a árvore
- O QUE USAR: aula10-bringup para a forma, aula11-mundo para os dados
- COMO ADEQUAR: o cenário tem de parecer com o ambiente do SEU robô — edite o mapa.py
- Os seus sensores precisam de frames com os nomes que os seus nós usam

Como saber que terminou

```
ros2 launch <seu_pacote> bringup.launch.py &      # aba 1, deixe rodando  
sleep 5
```

```
ros2 node list          # todos os seus nos aparecem?  
ros2 topic Hz /scan     # o sensor publica, e a que taxa?  
ros2 run tf2_tools view_frames  # UMA arvore, sem orfaos
```

Guarde as três saídas. Quem rodou e não guardou fez o trabalho e não provou.

Desafio: qual é o orçamento de deriva do seu projeto?

- Quanto erro de posição o seu projeto tolera antes de fazer besteira?
- Decida o número ANTES de medir — é a régua da Aula 9, noutro assunto
- Desviar de obstáculo tolera 1 m; contar frutos numa fileira tolera 20 cm
- Depois meça: `for d in 0.5 1 2 3 5; do python3 simulator.py --deriva $d ...; done`
- E responda: o que você faria se precisasse do dobro da distância?

Para a próxima aula, 06/10 — SLAM Toolbox

- A aresta map -> odom deixa de ser identidade e passa a ser calculada
- O mapa do próximo gate sai disso
- Chegue com o bringup entregue e a árvore de transformadas fechada
- O SLAM é construído sobre a TF — ele não conserta TF errada